

实验三：自动写诗

尧祥临 202518023406038 前沿交叉科学学院

1 任务介绍

本次实验的任务是基于预处理后的唐诗数据集训练一个字符级语言模型，并利用模型根据给定首句自动续写诗句。数据集以 npz 格式保存，包含诗句索引矩阵、ix2word 和 word2ix 三个部分。每首诗已经被转换为字典中的序号，特殊符号包括 <START>、<EOP> 和 </s>，其中 </s> 用于 padding。除了普通首句续写外，这里还单独实现了藏头诗生成脚本，并且使用固定句长和标点约束改善诗歌格式的稳定性。

2 数据处理

原始数据中每首诗长度固定为 125，不足部分使用 </s> 填充。为了使训练序列和生成时的输入格式一致，数据预处理时首先定位每首诗中的 <START>，去除其前面的 padding；随后找到 <EOP> 作为诗句结束位置，并保留从 <START> 到 <EOP> 的完整内容。如果序列超过最大长度，则截断并在末尾补上 <EOP>；如果不足最大长度，则在右侧继续填充 </s>。

这样的处理有两个目的。第一，模型在训练时能够明确学习从 <START> 开始生成诗句，到 <EOP> 结束诗句的过程。第二，padding 只出现在真实诗句之后，损失函数可以通过 ignore_index 忽略这些位置，避免模型把大量 padding token 当作正常预测目标。

```
1 def prepare_data():
2     datas = np.load(DATA_PATH, allow_pickle=True)
3     raw_data = datas["data"]
4     ix2word = datas["ix2word"].item()
5     word2ix = datas["word2ix"].item()
6
7     start_idx = word2ix["<START>"]
8     eop_idx = word2ix["<EOP>"]
9     pad_idx = word2ix["</s>"]
10
11     poems = []
12     for i in range(len(raw_data)):
13         row = raw_data[i]
14         start_pos = int((row == start_idx).argmax())
15         eop_positions = row == eop_idx
16         if eop_positions.any():
17             eop_pos = int(eop_positions.argmax())
18             content = row[start_pos:eop_pos + 1]
19         else:
20             content = row[start_pos:]
21
```

```

22     if len(content) > MAX_SEQ_LEN:
23         content = content[:MAX_SEQ_LEN - 1]
24         content = np.append(content, eop_idx)
25
26     padded = np.full(MAX_SEQ_LEN, pad_idx, dtype=np.int64)
27     padded[:len(content)] = content
28     poems.append(padded)
29
30 data = torch.from_numpy(np.stack(poems)).long()
31 return data, ix2word, word2ix

```

3 模型架构

本实验使用的模型由 Embedding、多层 LSTM、LayerNorm 和共享权重输出层组成。字符首先经过 Embedding 层映射到 512 维向量空间，随后通过 dropout 进行正则化，再输入两层 LSTM 建模上下文依赖。LSTM 输出后使用 LayerNorm 稳定隐藏状态分布，最后通过共享 Embedding 权重的线性投影得到每个位置上的词表 logits。

模型整体流程为

Input → Embedding → Dropout → 2-layer LSTM → LayerNorm → Weight Tying Output.

表 1: 自动写诗模型结构

模块	输出形状	说明
Input	$B \times L$	字符索引序列
Embedding	$B \times L \times 512$	<code>padding_idx</code> 使 padding 向量不更新
Dropout	$B \times L \times 512$	减弱训练阶段的共适应
LSTM	$B \times L \times 512$	2 层，隐藏维度 512
LayerNorm	$B \times L \times 512$	稳定 LSTM 输出分布
Shared Output	$BL \times 8293$	与 Embedding 共享输出权重

词表大小为 8293，模型总参数量为 8457829。Embedding 层包含 8293×512 个参数。由于输出层采用 weight tying，输出投影不再额外学习一个 512×8293 的独立权重矩阵，而是直接复用输入 Embedding 矩阵，只额外学习一个词表大小的偏置。这样一方面减少了约 425 万个输出层参数，另一方面也使输入字符表示和输出字符分类空间保持一致。对于字符级语言模型而言，同一个字既会作为输入上下文出现，也会作为下一字预测目标出现，因此共享输入和输出表示是比较自然的约束。

模型核心代码如下：

```

1 class PoetryModel(nn.Module):

```

```
2 def __init__(self, vocab_size, embedding_dim=300, hidden_dim=300,
3             num_layers=2, dropout=0.5, padding_idx=None):
4     super().__init__()
5     self.hidden_dim = hidden_dim
6     self.num_layers = num_layers
7
8     self.embedding = nn.Embedding(
9         vocab_size,
10        embedding_dim,
11        padding_idx=padding_idx
12    )
13    self.emb_dropout = nn.Dropout(dropout)
14    self.lstm = nn.LSTM(
15        embedding_dim, hidden_dim,
16        num_layers=num_layers,
17        batch_first=True,
18        dropout=dropout if num_layers > 1 else 0.0
19    )
20    self.norm = nn.LayerNorm(hidden_dim)
21    self.output_bias = nn.Parameter(torch.zeros(vocab_size))
22
23    def forward(self, x, hidden=None):
24        batch_size, seq_len = x.size()
25        embeds = self.emb_dropout(self.embedding(x))
26        if hidden is None:
27            h_0 = torch.zeros(self.num_layers, batch_size,
28                             self.hidden_dim, device=x.device)
29            c_0 = torch.zeros(self.num_layers, batch_size,
30                             self.hidden_dim, device=x.device)
31            hidden = (h_0, c_0)
32
33        output, hidden = self.lstm(embeds, hidden)
34        output = self.norm(output)
35        output = F.linear(output, self.embedding.weight, self.output_bias)
36        output = output.reshape(batch_size * seq_len, -1)
37        return output, hidden
```

4 训练方法

训练过程使用 teacher forcing。设诗句 token 序列为 $x_0, x_1, \dots, x_{L-1}$ ，训练输入为 $x_0, \dots, x_{L-2}$ ，预测目标为 $x_1, \dots, x_{L-1}$ 。对每个非 padding 位置计算交叉熵损失：

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(z_{i,y_i})}{\sum_{j=1}^V \exp(z_{i,j})},$$

其中 V 为词表大小， N 为 batch 中非 padding token 数量。困惑度由

$$\text{PPL} = \exp(\mathcal{L})$$

计算得到。由于训练阶段启用了 dropout，而验证阶段使用 eval 模式关闭 dropout，因此训练困惑度略高于验证困惑度是可以接受的，并不一定表示验证集划分异常。

优化器采用 AdamW，并使用前 3 个 epoch 线性 warmup、之后余弦退火的学习率策略。训练中加入梯度裁剪，防止 RNN 训练时出现梯度爆炸。验证集占全部数据的 10%，每个 epoch 结束后计算验证 loss 和验证困惑度，并保存验证 loss 最低的模型。

表 2: 主要训练配置

项目	设置	项目	设置
Batch size	128	Epochs	60
Embedding dim	512	Hidden dim	512
LSTM layers	2	Dropout	0.5
Optimizer	AdamW	Learning rate	1×10^{-3}
Weight decay	1×10^{-4}	Gradient clip	1.0
Max length	125	Validation split	10%

5 训练结果分析

表 3 列出了若干关键 epoch 的训练记录。训练初期模型从随机状态开始学习字符转移关系，第 1 轮验证困惑度仍高达 1340.5；到第 3 轮 warmup 结束时，验证困惑度下降到 309.3。随着训练继续进行，loss 和困惑度持续下降，最终第 60 轮验证 loss 为 4.5867，验证困惑度为 98.17。

表 3: 训练过程中的 loss 与 perplexity

Epoch	Train Loss	Train PPL	Val Loss	Val PPL	Grad Norm
1	17.1580	28289150.5	7.2008	1340.5	8.453
3	6.0091	407.1	5.7344	309.3	2.156
10	5.2267	186.2	5.0532	156.5	0.797
20	4.9798	145.4	4.8341	125.7	0.697
40	4.7265	112.9	4.6369	103.2	0.615
60	4.6504	104.6	4.5867	98.2	0.556

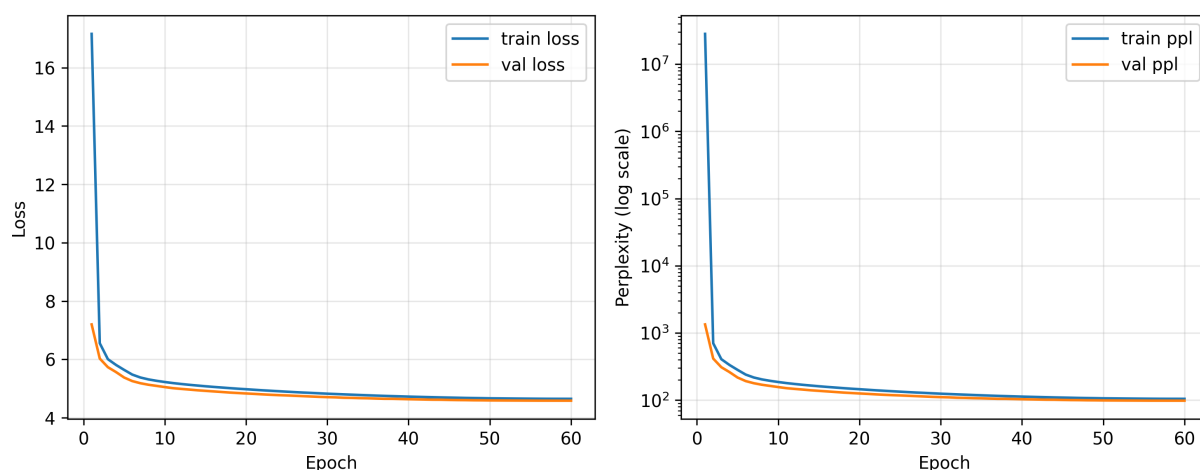


图 1: 训练集与验证集 loss、perplexity 变化曲线

从图 1 可以看到，loss 在前几轮下降最快，之后进入较慢的收敛阶段。右侧困惑度曲线使用对数纵轴，以避免第 1 轮极大的训练困惑度压缩后续变化。模型最开始几乎无法给出稳定预测，随后快速学习到唐诗语料中的高频字、常见词组和标点模式；训练后半段困惑度仍在缓慢下降，但改善幅度明显变小。这说明模型已经学到基本语言模式，但由于任务本身是开放式文本生成，同一上下文后可能存在大量合理后继字符，因此困惑度不会像分类任务中的 loss 那样下降到很低。

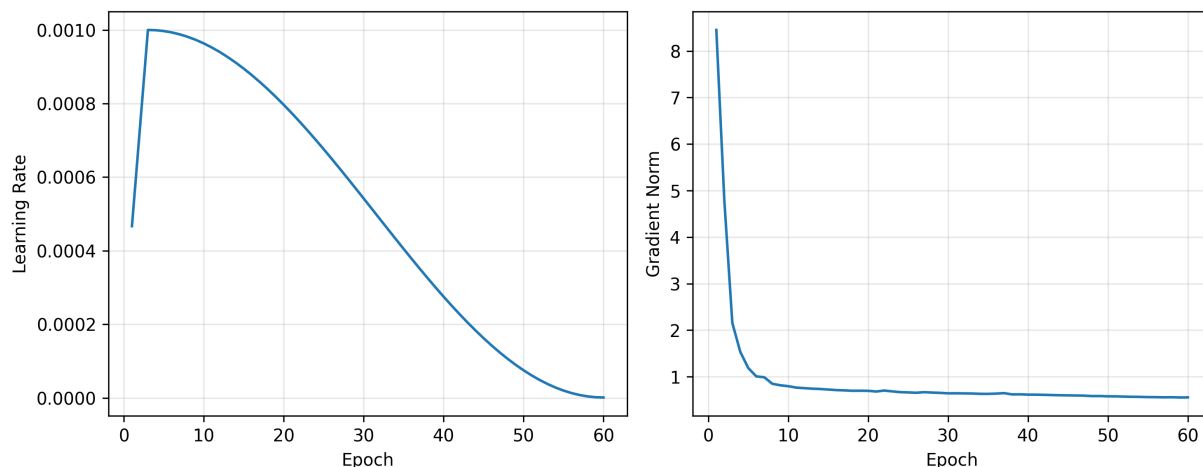


图 2: 学习率与梯度范数变化曲线

图 2 展示了学习率和梯度范数的变化。前 3 轮学习率逐步升高，帮助模型从随机初始化平稳进入训练；之后余弦退火逐渐降低学习率，使后期更新幅度减小。梯度范数在第 1 轮较大，随后快速下降并稳定在较小范围，说明梯度裁剪和学习率调度能够较好地控制 RNN 训练中的不稳定问题。

6 普通诗句生成

普通续写代码先将给定首句拆成字符列表，使用 <START> 初始化模型状态。生成前若干步时，不直接采样，而是强制将首句中的字符依次喂入模型，使隐藏状态吸收给定首句的信息；首句结束后，再根据模型输出 logits 进行 top-p 采样。相比直接取概率最大的字符，top-p 采样保留了一定随机性，可以避免生成结果过于单调。

```
1 def generate(model, start_words, ix2word, word2ix, device,
2             max_len=MAX_GEN_LEN, temperature=TEMPERATURE, top_p=TOP_P):
3     results = list(start_words)
4     input_tensor = torch.tensor([word2ix["<START>"]]).view(1, 1).to(device)
5     hidden = None
6
7     with torch.no_grad():
8         for i in range(max_len):
9             output, hidden = model(input_tensor, hidden)
10            if i < len(start_words):
11                w = results[i]
12                next_idx = word2ix.get(w, word2ix["<START>"])
13            else:
14                logits = output.data[0]
15                sorted_logits, sorted_indices = torch.sort(logits, descending=True)
16                sorted_probs = torch.softmax(sorted_logits / temperature, dim=-1)
17                cumsum = torch.cumsum(sorted_probs, dim=-1)
18                keep = cumsum <= top_p
19                keep[0] = True
20                filtered_probs = sorted_probs * keep.float()
21                filtered_probs /= filtered_probs.sum()
22                chosen = torch.multinomial(filtered_probs, num_samples=1).item()
23                next_idx = sorted_indices[chosen].item()
24                w = ix2word.get(next_idx, "<EOP>")
25                results.append(w)
26            input_tensor = input_tensor.data.new([next_idx]).view(1, 1)
27    return "".join(results)
```

以下为实际运行模型得到的普通续写结果：

首句：湖光秋月两相和

湖光秋月两相和，落日江南一半愁。

首句：白日依山尽

白日依山尽，青山万里平。天寒云外出，水阔楚山低。流水逢归客，春风入竹林。明朝更惆怅，更寄旧时人。

首句：海上生明月

海上生明月，还应忆九天。春风开露叶，玉树含芳菲。今日湖中月，同来旧思君。

这些输出在局部词组和句法上已经具有明显的唐诗风格，例如“落日江南”“青山万里”“春风开露叶”等短语比较自然。但模型仍然存在开放式生成中的常见问题：一是整首诗主题不一定始终集中，二是长距离语义递进较弱，三是有时会把较常见的意象反复组合。其根本原因在于模型只通过字符级语言建模学习局部上下文概率，没有显式建模诗歌主题、平仄和对仗结构。

7 藏头诗生成

藏头诗生成和普通续写的目标不同。普通续写只要求给定首句后继续生成，而藏头诗要求每一句的第一个字严格等于指定藏头字，同时还要保持五言或七言的固定句长。如果直接沿用普通生成策略，模型可能在一句中间提前生成标点，也可能生成长度不稳定的句子，导致五言、七言格式被破坏。因此我将藏头诗实现为一个单独脚本，在生成策略上加入格式约束。

具体做法是：先用 <START> 初始化隐藏状态；对藏头字符串中的每个字，强制把它作为当前句第一个字喂给模型；随后采样生成剩余 `line_len-1` 个字，其中 `line_len` 可以设置为 5 或 7；一句内部屏蔽 <START>、<EOP>、</s> 以及逗号、句号，防止模型提前结束或在句内输出标点；句末再由程序按照位置手动补上逗号或句号。整首诗生成过程中不重置 hidden state，使上下句之间仍保留一定上下文连续性。

```
1 def generate_acrostic(model, heads, ix2word, word2ix, device,
2                       line_len=LINE_LEN, temperature=TEMPERATURE, top_p=TOP_P):
3     model.eval()
4     for word in heads:
5         if word not in word2ix:
6             raise ValueError(f"'{word}' not in vocabulary")
7
8     poems = []
9     hidden = None
10    with torch.no_grad():
11        _, hidden = feed_word(model, word2ix["<START>"], hidden, device)
12
13        for i, head in enumerate(heads):
14            line = [head]
15            output, hidden = feed_word(model, word2ix[head], hidden, device)
16
17            for _ in range(line_len - 1):
18                next_idx, word = sample_word(output, ix2word, word2ix)
19                line.append(word)
20                output, hidden = feed_word(model, next_idx, hidden, device)
21
22            punctuation = "。" if i % 2 == 1 or i == len(heads) - 1 else ", "
23            line.append(punctuation)
```

```
24         if punctuation in word2ix:  
25             _, hidden = feed_word(model, word2ix[punctuation], hidden, device)  
26             poems.append("".join(line))  
27     return "\n".join(poems)
```

下面给出实际运行得到的藏头诗结果。可以看到，在固定句长和句内标点屏蔽后，每句首字和句式都比较稳定。

五言藏头：深度学习

深山幽径可，
度地隔溪深。
学院时时见，
习门声渐闻。

七言藏头：丰川祥子

丰年今日在京都，
川路何妨在白云。
祥水自从名不见，
子孙难见好相过。

七言藏头：千早爱音

千里云山万里愁，
早秋城郭旧时秋。
爱君何事君为乐，
音信谁知此意稀。

五言藏头：长崎素世

长安风雨老，
崎岖傍空岑。
素彩含长纓，
世上还可怜。

从输出结果看，藏头约束能够稳定控制每句首字，五言和七言句长也能由 `line_len` 直接控制。相比普通续写，这里的生成自由度更低，但格式更可靠。需要注意的是，强行指定首字也会带来副作用：如果藏头字之间语义关联较弱，模型只能在局部范围内补全每一句，整首诗的主题连贯性可能不如自然生成。如果要想实现更强的效果，可以考虑在藏头诗中加入主题词或韵脚约束，使格式控制和语义控制同时发挥作用，但在本实验就不过度展开了。

8 总结

本次实验基于 PyTorch 实现了一个字符级自动写诗模型，完成了数据预处理、Embedding-LSTM 语言模型构建、训练日志记录、普通诗句续写和藏头诗生成等流程。模型使用两层 LSTM 建模字符上

下文，在输出前加入 LayerNorm 稳定隐藏状态，并采用 weight tying 共享输入 Embedding 和输出投影权重。训练 60 个 epoch 后，验证困惑度下降到 98.17，模型能够生成具有一定唐诗风格的句子。

从实验结果看，模型已经能够学习到常见诗歌意象、基本句法和标点节奏，但仍然缺少对全局主题、对仗、格律和平仄的显式控制。普通续写更自然，但句长和结构不完全可控；藏头诗通过固定五言或七言句长、屏蔽句内标点并手动补标点，使输出格式更加稳定。后续如果继续改进，可以尝试引入更强的序列模型、预训练语言模型，也可以在解码阶段加入重复惩罚、韵脚约束，以进一步提升生成诗句的质量。